Handling Execution Overruns in Hard Real-Time Control Systems: Theory, Overhead, and an Image-Processing Prototype

Christian Percival
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany

***Abstract*—The worst-case execution time (WCET) is the common approach for analyzing hard real-time tasks. However, this method can lead to the inefficiency and waste of processor resources in cases when the worst-case scenario is infrequent. For example, this happens with the image search performed by an imaging system. In a regular situation, the object finding area is small, while in cases of complex searches, there may be need to scan the whole image. The paper contains reviews of execution overruns, rate adaptation, constant bandwidth server (CBS), and hard-deadline CBS variant, CBShd. The authors provide mathematical models of tasks and the bandwidth, discuss the deadline-recharge algorithm including the resource sharing. The python/openCV prototype measures the load of image search task depending on input and applies the same data in three simple models of scheduling including normal budget-only, worst-case time based, and time in case of overruns. The prototype, however, is not intended to be a kernel CBS implementation but demonstrates the balance between the normal speed of execution, deadline misses, and overruns.**

***Index Terms*—execution overrun, hard real-time systems, Constant Bandwidth Server, CBShd, WCET, EDF, rate adaptation**

I. Introduction

The validity of a real-time result is in question only when both its value and its time of accomplishment satisfy the requirements of the relevant system. In this case, a task that overruns its estimation budget may delay other tasks and consequently affect the validity of other results obtained due to the arrival of all subsequent tasks later than they should. This is known as *execution overrun*. It is important to distinguish overrun from overload. An event of overrun concerns a single task or process in operation while overload indicates that the total demand for the processors exceeds their available capacity.

In the classical theory of hard real time analysis, each task has its Worst Case Execution Time (WCET). This is reasonable if the variation of execution time is low, but is not optimal for visual sensing and perception tasks because in this case the runtime is mainly determined by the input. Caccamo, Buttazzo, and Sha illustrate the tracked visual objects as examples of this kind of task. An object is usually detected within its predetermined search window but an obstacle may require the search process to expand making it possible to carry out a frame scan fully and at worst many times [1]. So in this case, although most frames pose no difficulty, the requirement to reserve this time period in every cycle hinders the control frequency significantly.

The implementation and measurement data are available in the accompanying project and at the project repository.[1]

II. Mathematical Background

A. Task, budget, and deadline model

A periodic task is represented by

$$\tau_i = (C_i, T_i, D_i), \tag{1}$$

where C_i is a safe execution-time bound, T_i the activation period and D_i the relative deadline. In the overrun model, task τ_i additionally has a normal computation time c_i^n and a WCET, with

$$c_i^n \leq c_{i,j} \leq \mathrm{WCET}_i. \tag{2}$$

Job j overruns its normal allocation when

$$c_{i,j} > c_i^n, \qquad o_{i,j} = c_{i,j} - c_i^n. \tag{3}$$

An overrun is not automatically a deadline miss. A task may exceed its budget but still complete before D_i.

Under earliest deadline first (EDF), a task set of independent preemptive periodic tasks with deadlines equal to periods is schedulable on one processor if its utilization does not exceed one [2]:

$$U = \sum_i \frac{C_i}{T_i} \leq 1. \tag{4}$$

For a reservation server, the assigned bandwidth is

$$U_i = \frac{Q_i}{T_i^s}, \tag{5}$$

where Q_i is the server budget and T_i^s its server period. These are design parameters derived from application specific timing requirements. They are not derived directly from CPU clock frequency.

[1]Project repository: https://github.com/ChristianPercival/HandlingOverruns

B. Control-performance motivation

The main paper combines scheduling with rate adaptation. A representative performance-loss model is

$$\Delta J_i(f_i) = \alpha_i e^{-\beta_i f_i}, \tag{6}$$

where increasing the sampling frequency f_i reduces performance loss. The system therefore benefits from high normal-case frequencies, but each control task must remain above a minimum stable frequency $f_i^{\min}$ [1], [3]. For hard-deadline protection, the allocated bandwidth has to be sufficient for the worst case scinario, for example,

$$U_i \geq f_i^{\min} \mathrm{WCET}_i, \qquad \sum_i U_i \leq 1. \tag{7}$$

III. Overrun-Handling Algorithms

A. WCET and normal-budget baselines

The scheduling strategy based on WCET allocates a certain amount of time for execution in accordance with WCET_i for each task. When the estimates and system model are both accurate this gives deterministic and real time scheduling. However, for normal tasks much of processor capacity remains idle in this case. Conversely, scheduling based only on the normal budget uses c_i^n and increases the release frequency, but has no protection when the task consumes more time than it has in accordance with the budget when an overrun occurs.

B. Constant Bandwidth Server

CBS was introduced to allocate a guaranteed share of resources for tasks with variable execution needs while protecting hard real time work [4]. When using EDF, the server has a budget of Q_i, period T_i^s, current budget q_i and deadline d_i^s. During task execution current budget q_i decreases. After the budget is spent CBS restores it and delays the task deadline. Since EDF gives priority to tasks with earlier deadlines, tasks with an overrun have less urgency.

The key feature of CBS is temporal isolation. Tasks that need more share of allocated time perform slower without occupying unallocated time needed by other tasks.

C. Hard-deadline CBS variant

The CBShd algorithm in [1] uses an estimate $c_{i,j}^r$ of the current job's remaining computation time. At the point where the budget is used up, the update rule for the next budget time is:

$$\begin{aligned} \text{if } c_{i,j}^r \geq Q_i: &\quad q_i \leftarrow Q_i, \quad d_i^s \leftarrow d_i^s + T_i^s, \\ \text{else}: &\quad q_i \leftarrow c_{i,j}^r, \quad d_i^s \leftarrow d_i^s + \frac{c_{i,j}^r}{U_i}. \end{aligned} \tag{8}$$

Thus, a larger remainder gets the full recharge of the budget for the upcoming execution while a smaller remainder gets just the amount needed for the completion of the specific job. That is why unnecessary last deadline postponement is avoided whenever the required execution time is lower than the total budget. The estimation of the required execution time can be obtained from the WCET minus the time that the task has already consumed. As a result, WCET stays in the system as part of the hard-deadline argument even if the regular budget is lower.

D. Algorithmic overhead

The overrun handler implements execution time accounting, budget depletion tests, deadline changes, ready queue operations, and timer functions. Also accounting for possible preemption and cache interference. Caccamo and other authors found the value of about $42\,\mu$s per CBShd deadline postponement for a 133 MHz Pentium in the SHARK kernel system; that value is platform-oriented and excludes the preemption overhead [1]. Smaller budgets positively influence adaptation granularity. Nevertheless, they promote higher number of postponements, thus selection of parameters is necessary to achieve some compromise between control properties and runtime overhead.

The sharing of resources creates new risks. It can happen that during a lock, a server runs out of the budget allocated to it. Some bounded-blocking protocols such as the Stack Resource Policy (SRP) are present in the practice of guaranteed access to resources using the EDF algorithm [5]. Some modern reservation systems may have mechanisms for uptake or recovery of bandwidth in the case when reservations conflict with each other or their sizing has been done incorrectly [6].

IV. Prototype and Formal Model

A. Application example

The prototype embodies the gist of the concept of visual search expressed in the paper. The system is able to generate 120 synthetic images, where 30 of them are easy cases, 30 - medium, 30 - hard and 30 - cases of missing objects. The task of finding a red item is performed by searching in the inner predicted area, then looking for the item in the medium area, and finally searching throughout the entire frame. The timer in Python is responsible for measuring the total processing time. The measured data is analyzed afterwards with different scheduling assumptions in mind. The handlers do not alter the raw running time of OpenCV and just apply the principles of handling to running times of all tasks.

Figure 1 presents the strong dependence on the input. The average times for easy and medium frames are approximately 0.148 ms and 0.348 ms, while for difficult and missing cases they are 7.996 ms and 7.818 ms correspondingly. The maximum time recorded was 12.753 ms. The missing frames can be slightly faster than frames marked as difficult as both types require complete exploration but the behavior of an empty frame with respect to the threshold and caching mechanisms can differ significantly. Regular operating system and measurements jitter can contribute to this.

B. Scheduling experiment

The simple method utilizes an image budget of 0.5 ms, assuming the cycle time to be 2 ms. Thus, the image bandwidth is at 25% and the nominal throughput is at 500 Hz. The period does not arise from the CPU clock speed but is taken as an assumption for demonstration purposes. Any task lasting more

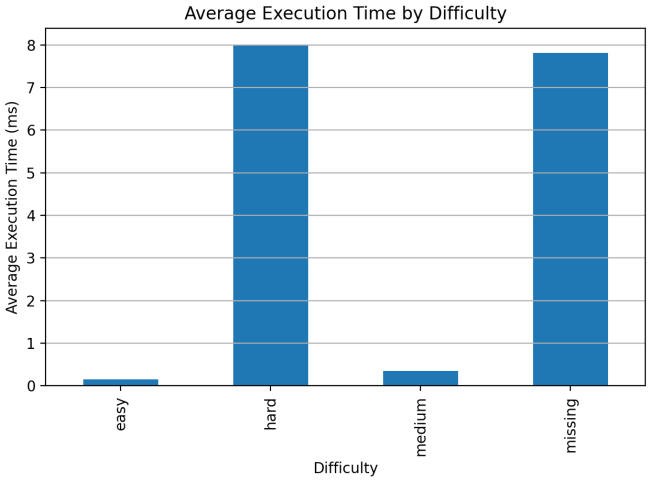

Fig. 1. Measured image-processing runtime by input difficulty.

Fig. 2. Deadline misses in the simplified scheduling comparison.

than 0.5 ms is considered as an overrunning task, while a standard task lasting longer than 2 ms is considered to violate its nominal deadline.

The three compared interpretations are:

1) **NormalOnly**: retain the 2 ms nominal deadline and count late jobs;
2) **WCET**: use a period equal to the measured maximum plus a 10% margin;
3) **OverrunHandled**: retain the normal-case release assumption but extend the affected job's allowed completion according to its excess work and reserved bandwidth.

The third item is CBS-inspired timing accounting, and not a kernel-level implementation or a proof of CBS/CBShd.

As for the analyzed processing times, NormalOnly fails to meet deadlines in 60 cases as opposed to zero deadline failures in case of maximum computation time and processed interpretations as seen in (Fig. 2). The nominal frequency of

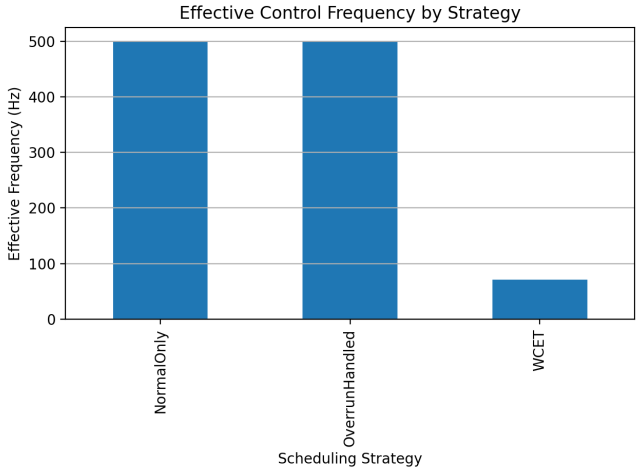

Fig. 3. Nominal task frequency implied by the selected periods. The 500 Hz bars are release assumptions, not guaranteed fresh-result rates.

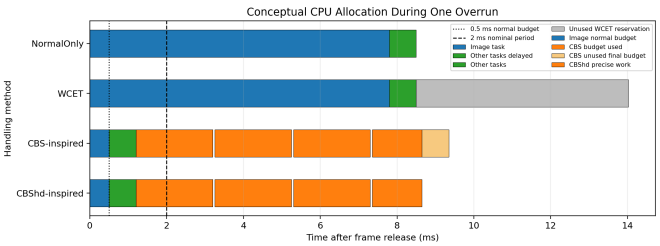

Fig. 4. Conceptual treatment of one 7.8 ms overrun.

releases is equal in both cases which is 500 Hz for normal time but is close to 71.3 Hz Hz for the maximum computation time case. Just because the two modes have the same nominal frequency does not mean they are equally successful, as NormalOnly appears to be unreliable in difficult cases.

The frequency comparison in Fig. 3 It highlights primarily the impact a WCET has on the frequencies due to the extended duration. It keeps NormalOnly definition with respect to NormalOnly and does not differentiate between the two definitions since both types of task utilize the same nominal periods of release. Through the behavior concerning the deadlines, it distinguishes between NormalOnly and the handled interpretation instead. A NormalOnly process will affect the period of subsequent jobs when executing a long process, but the handled affair will improve deadlines when the prolonged operation is being processed.

Figure 4 is meant to be understood in a conceptual way. NormalOnly is only responsible for delaying all other processes, WCET has the corresponding worst-case scenario saved, CBS represents complete recharge time blocks, while CBShd shows the number of units left over at the end. The image shows the difference between algorithms, rather than a specific trace of the kernel.

V. Discussion

The findings of this experiment are threefold. The first crucial finding states that, the need for execution dependent input is sufficient in creating overruns in various situations when regular cases are quite brief. The second conclusion states that WCET reservation becomes beneficial in case of very strict sequential chains that always have to produce a new result before moving to the next phase, assuming that WCET parameters together with the blocking/overhead estimates are credible. The conclusion draws attention to the idea that CBS or CBShd become most applicable in case of varying execution requests with the ability of a job to be locally delayed without causing the failure of the application, for example, for more complicated functions, like visual tracking, multimedia processing and robotics.

CBS/CBShd cannot be used instead of the traditional WCET analysis in case of strict time deadlines of high importance for the first phase of execution. Strictly speaking, it requires a bounded algorithm for all situations, reservation of sufficient capacity, equipment of needed hardware acceleration together with resource allocation as well as making sure about the schedulability of the whole path from the sensor to the actuator. At the same time, if it is acceptable to have certain changes in the response time, resource reservation avoids any need to pay WCET for all jobs.

The prototype displays key drawbacks. Python and the host OS fail to be classified as hard real-time systems, the measured maximum cannot be deemed as certified WCET; the used mechanism alters deadlines offline and does not implement the CBS inside the scheduler; and cache misses, interrupts, context switches, blocking, and latency of the timer are not taken into account. Therefore, the outcomes are only proofs of concept.

VI. Conclusion

When in execution overrun, a task will be treated as a task that exceeds its cost in terms of time. It is important to consider this in order to avoid excessive amounts of postponement. Although it is possible to reserve WCET at every cycle in order to limit the frequency of useful control, CBS offers a better solution in the form of budgets and deadlines based measures that are taken by means of EDF scheduling. Another method, CBShd, is used at the final stage of the recharging process to make use of all computer resources still available.

REFERENCES

[1] M. Caccamo, G. Buttazzo, and L. Sha, "Handling execution overruns in hard real-time control systems," *IEEE Transactions on Computers*, vol. 51, no. 7, pp. 835–849, 2002, Main reference. Introduces local overrun handling, rate adaptation, CBShd, formal guarantees, overhead measurements, and an application-level chunked visual-search implementation. DOI: 10.1109/TC.2002.1017700.

[2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973, Foundational fixed- and dynamic-priority schedulability results, including EDF utilization arguments used in the background section. DOI: 10.1145/321738.321743.

[3] D. Seto, J. P. Lehoczky, L. Sha, and K. G. Shin, "On task schedulability in real-time control systems," in *Proceedings of the 17th IEEE Real-Time Systems Symposium*, Connects control performance, sampling frequency, and schedulability; provides background for frequency optimization and rate adaptation., 1996, pp. 13–21. DOI: 10.1109/REAL.1996.563699.

[4] L. Abeni and G. Buttazzo, "Integrating multimedia applications in hard real-time systems," in *Proceedings of the 19th IEEE Real-Time Systems Symposium*, Introduces the Constant Bandwidth Server approach for isolating variable soft and multimedia workloads from hard real-time tasks., 1998, pp. 4–13. DOI: 10.1109/REAL.1998.739726.

[5] T. P. Baker, "Stack-based scheduling of realtime processes," *Real-Time Systems*, vol. 3, no. 1, pp. 67–99, 1991, Introduces the Stack Resource Policy for bounded blocking and predictable resource sharing under EDF and fixed-priority scheduling. DOI: 10.1007/BF00365393.

[6] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed. Cham: Springer, 2024, Textbook background for EDF, CBS, overload and overrun handling, resource reservations, overhead, and resource-sharing protocols. DOI: 10.1007/978-3-031-45410-3.